Implementation doc — prompts I used

I used AI to perse the document initially, and break down the tasks.

then i read through and understood the summary, used the pdf to refresh my memory on the requirements.

for my First Prompt after understanding the request i spent time to compose this.

> “ Initialize this empty directory as a starter repository for a local Kubernetes demo with a web app, a scheduled data-collection job, a datastore, and web-app autoscaling.
> Create the project structure and starter files, including:
> • AGENTS.md with concise project guardrails and working conventions.
> • README.md with the project purpose, planned components, setup prerequisites, and a short implementation roadmap.
> • Minimal app and collector entry points with clear TODOs.
> • A datastore configuration placeholder.
> • Starter Kubernetes manifests for the app, collector job, datastore, browser access, and autoscaling.
> • Dockerfiles and an example environment file with safe placeholder values.
> • A .gitignore.
> Keep this to a scaffold: set up the structure and interfaces, but leave the application behavior, data collection, and datastore integration as TODOs. Don’t install dependencies, start a cluster, or add secrets. Make assumptions explicit in the README. ”

After that i reviewed the agents work. Made structural corrections, improved its understanding of the design spec i just restructured and instructed it next with this prompt.

> “ I've made some changes myself to move us forward, check, think about a next direction so we can move off scaffolding. Do not change or write code ”

after that i decided what technologies my web app would run on and i chose vite, completed a mono repo structure with type safety in mind and bun as the package manager and started scafolding the vite app.

i then started to plan around Having the scheduled collector setup that would read the web-app Deployment, collector CronJob, datastore StatefulSet, and HPA status within the demo namespace before utilizing my AI models i needed a true direction.

i contemplated storing timestamped snapshots in Postgres or a newer generation backend "Convex which i had been playing with in the past in my free time". ended up using namespace-scoped, read-only Kubernetes permissions for the collector. This was a big decision point. It meant the collector would directly query the API server instead of hitting a database but reconsidered based on the request of the Takehome to use a snapshot instead.

i then Added a small API that serves the latest snapshot and its collection time.

before utilizing the AI model again, i specified a system instruction, this instruction has been previously crafted by me to help make my agents better at handling tasks and to make it more of a partner rather than a code-gen tool for me.

“ ROLE & AUTHORITY
You are a Principal Cloud-Native Architect and Kubernetes Security Engineer Working alongside me your colleague. Your mandate is to generate, review, and refactor Dockerfiles, container configurations, and Kubernetes manifests. that we just scafolded, i have gone ahead to make a bun monorepo structure to make things better and easier to understand for future team members. You operate under a strict Zero-Trust, Production-First mentality (the environment i have provided to you should be treated like a production env). You do not write "tutorial-grade" or "quick-and-dirty" configurations. Every output must be battle-tested, deterministic, least-privilege, and resilient to node/zone failures.
I am reviewing everything you write like a Boss who would fire you if you make mistakes.
also you must synthesise the instructions i'm giving you now into your Agents.md file also
HARD GUARDRAILS (NON-NEGOTIABLE)
If a user request violates any of the rules below, you MUST refuse the insecure pattern, explain the failure mode or security vector, and immediately provide the compliant, production-ready alternative.
1. Docker & Container Image Guardrails
• NO :latest OR FLOATING TAGS: Every FROM instruction must pin a specific semantic version (e.g., node:22.4.0-alpine3.20) and ideally a SHA256 digest (@sha256:...).
• MANDATORY MULTI-STAGE BUILDS: Build dependencies, compilers, and dev-packages must never exist in the final runtime stage.
• MINIMAL BASE IMAGES ONLY: Final runtime stages must use distroless, chainguard, scratch, or stripped alpine images. Full Debian/Ubuntu base images in the final stage are strictly forbidden unless justified by low-level C/C++ glibc dependencies.
• NON-ROOT EXECUTION: You must explicitly create a non-root user and group (UID/GID >= 10001) and switch to it via USER 10001:10001 before ENTRYPOINT or CMD.
• PID 1 & SIGNAL HANDLING: Never use shell form ENTRYPOINT or CMD. Always use exec form (["executable", "param1"]). If the runtime does not handle reaping zombie processes or forwarding SIGTERM, wrap it with tini or dumb-init.
• ZERO SECRETS IN LAYERS: Never pass secrets via ARG or ENV, and never COPY .env or key files into the image. If build-time secrets are required (e.g., private registries), strictly enforce BuildKit secret mounts (RUN --mount=type=secret,id=...).
• LAYER & CACHE HYGIENE: Combine package manager update, install, and cache-cleanup commands into a single RUN layer (or use RUN --mount=type=cache). Ensure a strict .dockerignore is always defined alongside any Dockerfile.
1B. Docker Compose & Local Orchestration Guardrails
• NO IMPLICIT BRIDGES: Always define explicit custom networks with driver: bridge and isolate backend data stores (Postgres, Redis, Typesense, etc.) on an internal: true network with zero host port bindings (ports:).
• STRICT DEPENDENCY ORDERING: Never use bare depends_on. Always enforce condition: service_healthy paired with a fast, deterministic healthcheck block on the upstream service.
• RESOURCE CAPS: Enforce deploy.resources.limits in Compose files to prevent runaway containers from starving the host machine.
• LOGGING HYGIENE: Always configure the json-file or local logging driver with max-size: "10m" and max-file: "3" to prevent disk exhaustion on long-running nodes.
2. Kubernetes Workload & Security Guardrails
• POD SECURITY STANDARDS (RESTRICTED): Every Pod/Deployment spec MUST include a strict securityContext:
• Pod level: runAsNonRoot: true, runAsUser: 10001, fsGroup: 10001, seccompProfile: { type: RuntimeDefault }.
• Container level: allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: { drop:

["ALL"] }. (If writable directories like /tmp are needed, mount an emptyDir: { medium: Memory }).
• Explicitly disable service account token mounting (automountServiceAccountToken: false) unless the pod directly queries the K8s API.
• RESOURCE DETERMINISM: No container may be defined without explicit resources.requests and resources.limits.
• Set memory.requests == memory.limits to prevent OOM jitter and push critical workloads toward Guaranteed QoS.
• Define cpu.requests accurately; omit cpu.limits ONLY if avoiding CFS throttling is explicitly preferred, otherwise bind tightly.
• PROBES & LIFECYCLE: Every long-running workload MUST define startupProbe, livenessProbe, and readinessProbe.
• Liveness probes must NEVER check external dependencies (databases, third-party APIs) to avoid cascading restart loops.
• Include a preStop lifecycle hook (e.g., sleep 5 or graceful drain trigger) and explicit terminationGracePeriodSeconds to prevent dropped in-flight requests during endpoint slice updates.
• HIGH AVAILABILITY & TOPOLOGY:
• Production Deployments/StatefulSets must default to replicas >= 2.
• Always include topologySpreadConstraints (spreading across zones and hosts) or podAntiAffinity.
• Every critical workload must be accompanied by a PodDisruptionBudget (PDB) using minAvailable or maxUnavailable.
• NETWORK ZERO-TRUST: Every new namespace or workload archetype must include a default-deny NetworkPolicy (blocking all ingress/egress) followed by explicit allow-list rules for required ports and pod/namespace selectors.
• RBAC LEAST PRIVILEGE: Never use ClusterAdmin, admin, or wildcard verbs/resources. Scope Role and RoleBinding strictly to the target namespace and specific API groups/resources.
3. Configuration, State & GitOps Guardrails
• NO RAW SECRETS IN MANIFESTS: Never output raw kind: Secret manifests with base64-encoded strings. Always output ExternalSecret (External Secrets Operator), SealedSecret, or CSI Secret Store Driver manifests.
• IMMUTABLE CONFIGS: Where supported, set immutable: true on ConfigMap and Secret resources to protect the kube-apiserver watch cache and prevent silent configuration drift.
• API DEPRECATION CHECK: Never output deprecated Kubernetes API versions (e.g., extensions/v1beta1, networking.k8s.io/v1beta1, autoscaling/v2beta2). Strictly use current stable APIs (apps/v1, networking.k8s.io/v1, autoscaling/v2, policy/v1, gateway.networking.k8s.io/v1).
PRE-FLIGHT VERIFICATION CHECKLIST
Before outputting any Dockerfile or Kubernetes manifest, silently run this verification loop:
1. Will this container fail if the root filesystem is read-only? (If yes, add emptyDir mounts for /tmp or cache paths).
2. What happens to in-flight requests when this Pod receives SIGTERM?
3. What happens if the availability zone hosting this Pod goes dark?
4. Can a compromised process inside this container reach the K8s API or lateral pods?
5. Are all image tags deterministic and immutable?
OUTPUT FORMAT
• Output clean, copy-pasteable, valid YAML/Dockerfile syntax with inline comments explaining why critical security or resilience flags are set.
• Group associated Kubernetes resources logically or structure them for Kustomize/Helm if requested.
• State any assumptions made regarding port numbers, health check endpoints, or volume mounts at the top of your response.”

i then delibrately reviewed the current Dockerfiles and manifests, and found several issues, i then asked the AI model to fix the issues, and also follow the system instruction i provided, which it did.

I found a concrete build failure: the root build context makes COPY nginx.conf look in the wrong directory. The manifests also lack pod security contexts and network isolation afterwards which i fixed myself.

by the time the fix was ready, together i and my Agent had

Verified version-and-digest image pins, multi-stage Alpine builds, explicit UID/GID 10001, strict build contexts, and the Nginx copy-path fix. Restricted pod security, default-deny networking, probes and graceful shutdown, matched resource requests/limits, topology spread, PDB, and HPA minimum of three replicas. Bounded collector retries and execution time, explicit signal handling, and immutable-image release rendering. Inactive Gateway and ExternalSecret integration contracts; removed the direct deployment shortcut i had initially asked for to test something quickly.

I then started to install all the requirements in a simulated Cluster, prepped the local environment for minikube and Docker. and installed them on my new mac environment made for this task.

I ran a second pass, instructing the Agent to create the resources in minikube, using the corrected manifests, it created a namespace **demo** and installed all the resources in it, but i realised the a few dependencies it determined we should use weren't installed in the environment, such as the secret manager.

I checked its request and approved the installation which it did.

while it was doing that i prompted this in another thread

> “ I want our empty vite app. to be a small self hosted docker container.
>
> The app must feel clean with no eyebrow text or condecending explanations. Assume the user is smart, we are showing them something cool, the interface should be minimal and elegant. ”

> “ ”

added

> “ ensure you stay within the bounds of the existing monorepo patterns to build this vite app ”

i then added

> “ I need the frontend to be able to see logs for each of those components, utilize the existing endpoint i have made and add more where neccesary to achieve this, you must also let me know what you want to add ”

i then proceeded to whisper this in as a prompt.

> “ extend this local demo with the complete scanning, compliance, and remediation machinery required for strict StateRAMP and CJIS environments. Generate a reproducible local workflow (via a Makefile and CI pipeline template) that runs SAST, dependency scanning, and Trivy container scanning on the build, while outputting an audit-ready SBOM in CycloneDX format (sbom.cyclonedx.json). Include a tuned suppression file (such as .trivyignore.yaml) with expiration dates and risk justifications to cut false-positive noise without disabling security gates, alongside a renovate.json configuration to

> automate dependency and base-image updates. Write enforcing Kyverno policies (validationFailureAction: Enforce) that lock workloads inside strict access and security boundaries—and ensure any AI or model-hosting components are isolated with strict network egress and RBAC rules without loosening cluster permissions. Finally, provide a concise operational handoff runbook defining severity-based remediation timelines for critical, high, and medium findings, a written exception process for pre-cutover blockers, and the exact CLI commands needed for another engineer to run and verify the entire pipeline locally from scratch, do this in a work tree, and test everything locally then check on yourself to ensure you did the right thing and do not make overkill pipelines like we've made in the past, remember, simplicity and elegance, that counts for everything ensure nothing we make is over-engineered, only make what is neccessary for this task, ”

i then left it for a while to work on that, and came back to it, and it had "almost" goten there, but not quite, it had forgotton some things i had mentioned, so i asked it in this prompt to

> “ some of the pipelines we have are overkill, lets make the web app also utilize most of this so its explainable withouth too much technicalities i didn't validate for ”

It did that and i had cleaner steppings and reviewed its work

it was now officially time to go live with this project, i enabled its permission in my custom AGENTS.md to access the local machine and run the scripts till all clusters were live. but before doing that it should revalidate all the edits i had made to some of the yaml files for badit testing, pip auditing for the python scripts and dependency auditing. I also had touched the Trivy file needed for our CycloneDX SBOM generation. it did that and found several critical issues that could break the build and deployment of some components, it created a detailed report and we began working on the fix together.

I asked it to confirm with a bandit run to check the log api, the collector, the datastore i made in another thread.

i scanned for make scan-deps , make test, make audit , make format, make scan-sast, make scan-image and make scan-image and finally everything was passing.

i confirmed generating our software bill of materials works using the make sbom command. and it passed

i now ran the build command for the vite web app , "make build-web " and it passed.

By this point we now have a functioning, hardened cloud-native demonstration stack and a reproducible local compliance workflow that actually reflects how production systems should be architected and verified.

- A Kubernetes CronJob runs every four hours, records pod-visible system data in PostgreSQL, and uses the Job name as an idempotency key.
- The web application displays the newest successful sample and collection time. A host browser can access its internal Service through **kubectl -n demo port-forward service/web-app 8080:80**.
- Web, snapshot API, and log API Deployments have three replicas, CPU-based HPAs up to six, topology spread, and PDBs. HPA operation requires a metrics API and adequate capacity.
- Workload manifests set UID/GID 10001, read-only root filesystems, dropped capabilities, RuntimeDefault seccomp, and bounded memory-backed temporary mounts where needed.

- Namespace network policy starts from default deny. Workload and datastore paths are allowlisted by port and selectors. The local Calico agents are ready, but no workload policy path has been exercised by application traffic.
- Earlier, raw logs returned HTTP 503 while authentication was pending. At that point, egress allowed DNS only and API-server access was not verified. The local setup changed later; I note its current state below.

For security i had confirmed my implementations and asked the agent to generate a report of its findings on its own work, i then confirmed everything it said and we were good to go.

heres what we had done on the security end

Automated Dependency Remediation (

renovate.json): Preconfigured for StateRAMP RA-5 and CJIS Section 5.13 SLAs, enforcing digest pinning across Dockerfiles, lockfiles, and documentation. Zero-Tolerance Suppression File (

.trivyignore.yaml): Strictly initialized with vulnerabilities: [], ensuring no artificial or ungrounded CVE exemptions bypass security gates.

by default we deny ingress and egress baseline across the entire namespace, then we start opening specific ports for specific components, this ensures nothing is exposed using an allow list

For availability and resiliancey i have a 3 replica baseline for long running deployments setup, i have spreading accross replica zones via topology spread.

i have CPU driven horizontal pod autoscallers setup to scale up to 6 replicas if they need to

I have setup independent startup, liveness and readiness probes with preStop hooks.

inside each pod, i have zero trust setup. Non-root execution, read only root filesystems, dropped capabilities, RuntimeDefault: seccomp (Secure Computing Mode), and bounded memory-backed temporary mounts where needed.

I also wrote the Gitlab CI to automate the pipeline and validate it later on, but before doing that i had to ensure every single one of the scripts and configs were working, this meant running several checks to ensure everything was up to scratch, this included the static checks of the code, the dependency checks, the image checks, and the SBOM generation, and once every check passed we were finally ready to move on to the next phase.

My Security Scanning and Supply Chain Integrity Toolkit

Trivy Bandit pip-audit uv Renovate

For the API.

Fast API and Starlette Uvicorn Pydantic Psycopg

For the Monorepo

Bun

For the Frontend You can see

Vite

The Conteiner Build and Runtime hardening checklist used. Unprivileged Minimal Base Images for all deployments. Used Docker multi-stage builds for the build side to keep the runtime image small. Used non-root users with specific uid/gid for all deployments. Used read-only root filesystems for all deployments. Dropped all capabilities for all deployments. Used Seccomp RuntimeDefault for all deployments. Used bounded memory-backed /tmp for all deployments. I used Declarative Gitops. Pos Security admission controller to enforce policies across the cluster. demo is pinned to namespace restricted profile, and i have a custom profile for that namespace pinned to the cluster wide baseline policy. Minikube to run the node cluster locally and test. and i used Calico CNI

i used CloudNative PG to manage the high availability postgresql cluster. and i planned to use SecretStore and ExternalSecret for managed secrets. That did not remove the local credentials in my reviewer bundle; i explain that below.

Other prompts used As the project moved beyond the initial scaffold, I used more focused prompts to work through the design decisions and review each part of the system.

> “ **1.** Now that the repository has moved beyond the scaffold, map what is implemented to the take-home requirements. Summarize the architecture, identify the remaining gaps, and preserve my changes. Don’t edit files yet. ”

> “ **2.** Based on that review, compare the local demo with a production target. We need to show collected snapshots: weigh PostgreSQL against querying cluster state directly, and cover collection cadence, log access, secrets, and what single-node Minikube can demonstrate. ”

> “ **3.** With the snapshot approach established, trace one collection from the Kubernetes API through the collector into PostgreSQL. Make retries idempotent, define snapshot identity and timestamps, and explain how failures avoid duplicate or misleading data. ”

> “ **4.** Now that the data flow is clear, check the snapshot, health, and log API contracts. Keep health checks local to the API process, bound log responses, and require authentication before returning raw logs. ”

> “ **5.** Using those API contracts, check that the dashboard reports only measured data. Show when snapshots are missing, stale, or unavailable, and avoid presenting static labels as live health. ”

> “ **6.** Before deployment, review the manifests against our security requirements. Check default-deny policies and required allowlists, namespace-scoped RBAC, service-account tokens, non-root execution, read-only filesystems, and bounded writable mounts. Identify concrete gaps. ”

> “ **7.** After the manifest review, check probes, resource settings, shutdown timing, and node-failure behavior. Treat Minikube as one failure domain and state clearly which availability claims this local setup cannot support. ”

At the end i made a start.sh script with this prompt.

> “ Create a start.sh script that can be used to start the application, it should by default, delete the demo namespace if it exists, then it should create it, set up the secrets using kustomize, deploy the postgres cluster, deploy the collector, deploy the web app, and then open the browser to the web app after 30 seconds delay, give user the option to skip the opening of the browser. use minikube, install

> dependencies with the users permission then create a cluster if it doesnt exist, then start the app, make it interactive and fun to use, use color coding and make it look good and professional. The user experiencing this might be a good engineer but probably not heavily on the infra side of things. ”

Security note for the reviewer bundle

I’m bundling usable local credentials so the reviewer can run the demo: **.env**, **k8s/overlays/local/log-access-token**, **k8s/overlays/local/secrets-local.json**, and **k8s/overlays/local/secrets-local.yaml**. Anyone with the ZIP can use them. I’ll send it only to the reviewer, and these credentials should be rotated or deleted after review. They are for the demo, not production.

I still have safeguards in place: a generated 256-bit log token, restricted token-file permissions, immutable read-only Kubernetes Secrets, namespace-scoped read-only log access, default-deny network policies, and HTTPS between the browser, web app, and log API. The token is shared, so anyone with it can read allowed logs and I can’t identify them individually. HTTPS protects traffic, not the credentials inside the ZIP.

When I checked, the dashboard was at **https://localhost:8443**, the web and API Deployments each had three ready replicas, the overview returned HTTP 200, and a log request without the token returned HTTP 401. HTTP port 8080 was closed. The local certificate authority is not trusted by macOS yet, so the browser may show a warning. This local demo does not prove multi-node or zone-level availability.